

Improvements on KWS Model Development

Description: Describes the work done to improve the accuracy and robustness of the Keyword Spotting (KWS) demo application running on MAX78000/2 microprocessors.

Date Started: 07 Aug 2023

Date Ended: 09 Oct 2023

Purpose

Our main purpose in this application is to convert the data loader to a dynamic one. The previous data loader applies augmentation and saves the data to disk. This results in a one particular dataset. Our goal is to apply augmentation each time

data from the training set is called and randomise the dataset using this technique. To achieve this goal, we implemented some functions and changed the previous data loader. We tested our implementation and the discussed the results in the

discussion part.

Previous Data Loader

The previous data loader was designed in a particular order. The first thing is that the data files must be checked. In case of their existence, a dataset called '*dataset2.pt*' will be created to store the processed files. If the files are not downloaded, then

the download operations will be executed. After this, each audio file under each label will be converted to numeric values, and augmentation will be applied to all of them. Raw data and two more augmented data files will be saved for each audio.

This triples the current dataset. Inside the initialization, the saved dataset will be loaded and used for training and testing accordingly. One significant point here is that the augmentations are also applied to test data because the train-test split will

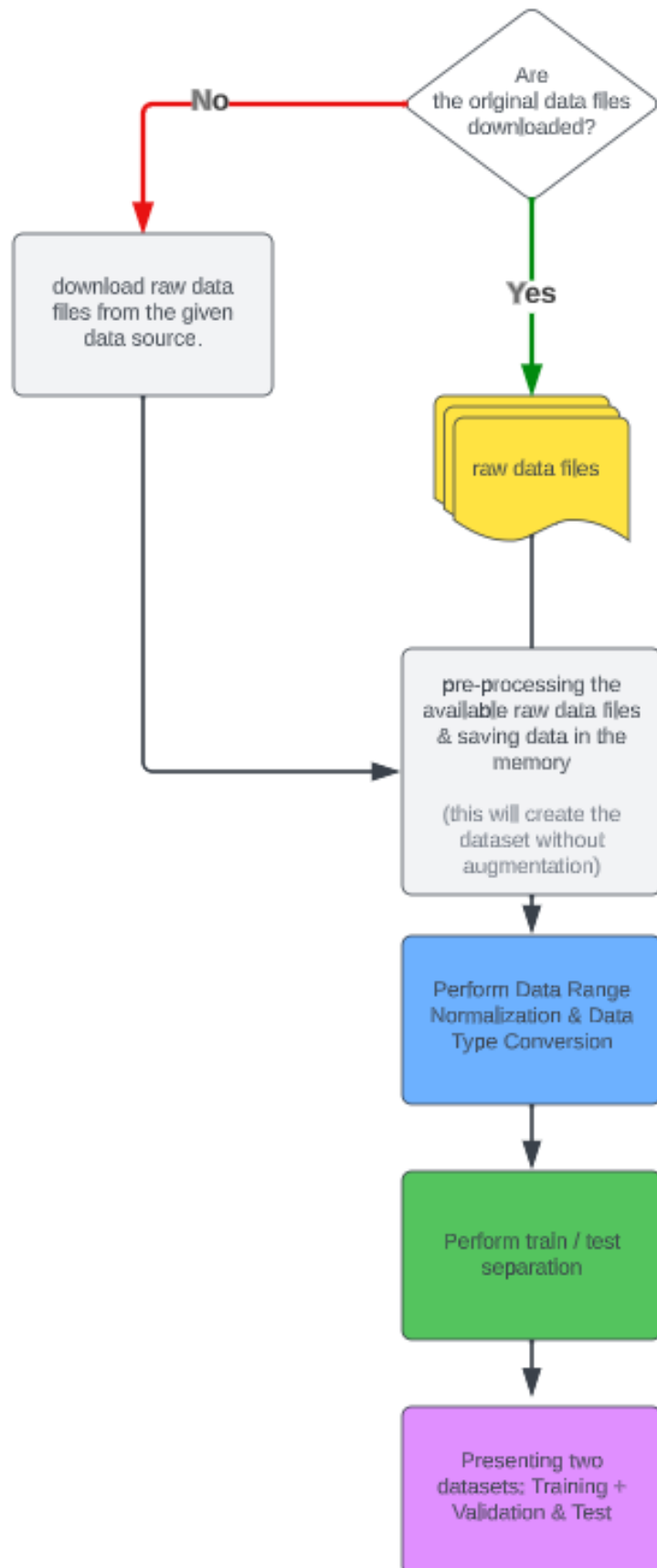
happen after the augmentations are applied to the raw data.

Dynamic Data Loader Application & Enhancements

To create a dynamic data loader, the first thing we do is create a dataset file for the augmented dataset. Instead of creating a file, we save them in memory after preprocessing the audio files. In this chapter, each change in our implementation will be

examined in detail. The main flow of the dynamic data loader can be seen in the next figure.

KWS Data Loader Main Flow



Create Raw Dataset

To create the raw dataset, the augmentation is removed from the `__gen_datasets` function. For each label inside the downloaded data, an empty numpy array is created for saving the data. This function takes all the audio files one by one and converts

them to numbers. The `data_class` array holds the associated labels for the audio files. The `data_type` array holds the appropriate number for training-test split for each record. After reading all the audio files with their labels, they are combined to create

full data, type, and class arrays. The resulting 3 arrays are added to create `raw_dataset`, and the `__gen_datasets()` function will return this dataset. When the `__init__()` function is called, this process will be executed, and the resulting dataset will be saved

in memory for later usage.

Array Name	Shape
data_in	(number of records inside a label, 16384)
data_type	(number of records inside a label, 1)
data_class	(number of records inside a label, 1)

The number 16384 is chosen because the input shape of the KWS model is (128, 128). After reading the audio files with a sampling rate of 16000 Hz, they are padded with 0 to make all the audio file sizes the same, which is $128 \times 128 = 16384$. Training,

validation, and test set split will be held using the data file names. The hash function is used to achieve a 90:10 split. Train data samples are labeled as 0, and test data samples are labeled as 1 in the `data_type` array described above.

Dynamic Augmentation & Reshaping

Application

Dynamic augmentation is applied each time the `__getitem__()` function is called. This makes the data loader dynamic and results in different data augmentations for each audio file during training. Unlike the previous data loader, the model sees a lot of

different augmentations for each audio, and this helps model achieve higher accuracy. After applying time stretch, signal shifting and noise to the coming audio file, the file must be reshaped before training in order to get the correct input shape for the

KWS model. After reshaping, `__getitem__()` will return (128, 128) data and the appropriate class label.

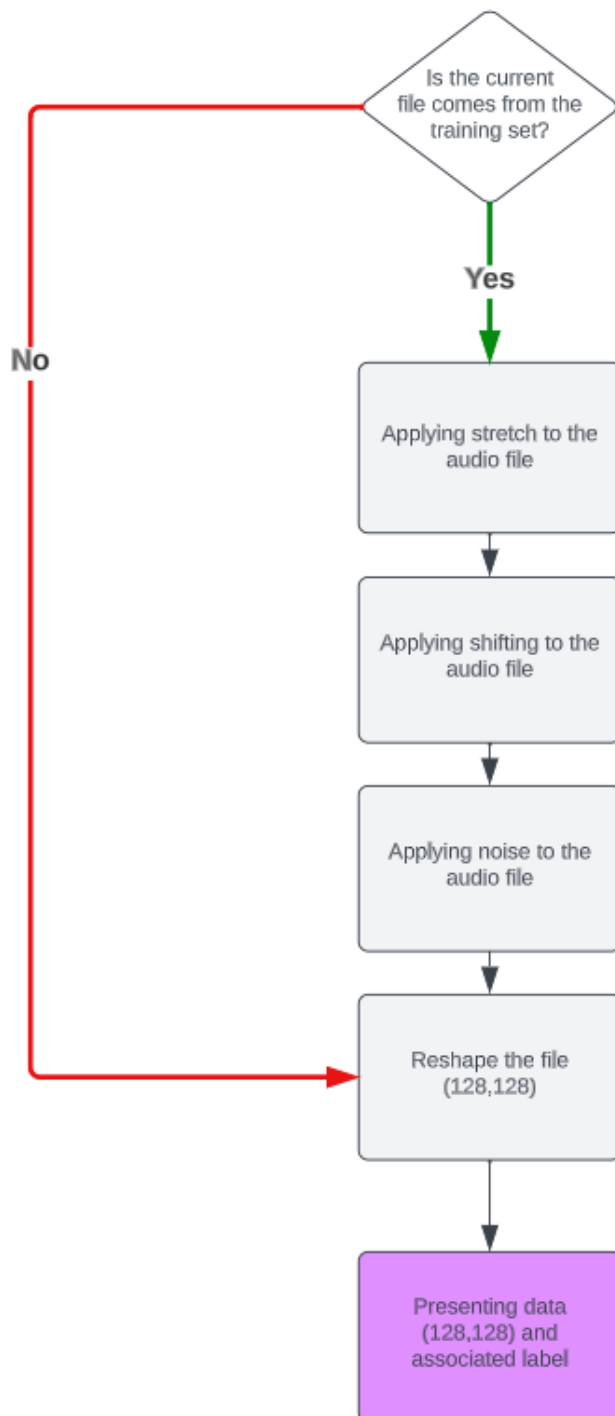
Here, the important part is that we do not apply augmentation techniques to our test set, unlike the previous data loader, which creates the augmented dataset and then split it into a training and test set. Our implementation creates the raw dataset and

splits it. To understand whether the data inside the `__getitem__()` function is from a training set or test set, we use the `data_type` array, which was described before. If the data is from a test set, we only apply reshaping in order to achieve the correct input

shape.

You can see the main work flow of `__getitem__()` function below.

__getitem__()



We also implemented an additional algorithm to make the augmentations more random. With this option, three augmentation techniques can be applied with a chance of 66% . This makes the training-validation data more versatile and varied, which can be

beneficial in long trainings.

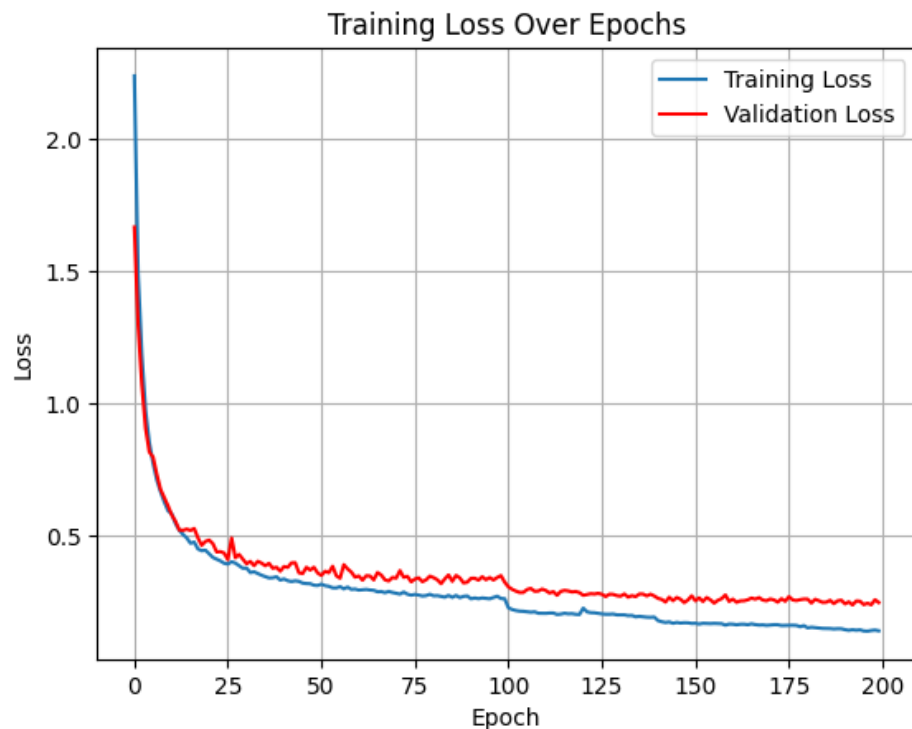
Inside the reshaping function, the augmentations may change the size of the data files. Since we want all of them to be (128, 128), we need to apply padding with 0 here. Padding is also done inside the reshape function.

Model Training

The training is done using [ai8x-training](#) repository and train.py script with recently arranged datasets/kws20.py script. The selected model was the CNN based ai85-kws20net as before in Model A. The training data contains around 100.000 samples and iterated over 200

epochs with two different schemes. First (Model B) is fully processed data with noise, time shift and time stretch and the latter (Model C) is the 66% chance of application of any augmentation which also can contain raw unprocessed data to be fed into the model. The learning

rate is set to 0.001 and no early stopping mechanism is implemented. The plot of the best training performance (Model C) in terms of training loss versus epochs can be seen below:



Evaluation and Discussion

After training two different models with brand new DataLoader with different augmentation techniques, we have tested our new model with both former and our DataLoaders. Therefore we have evaluated each different results of these combinations to verify

improvement achieved by our updates and modifications on the DataLoader script.

Strategy A: This model is trained with 3x more data and tested on augmented test data.

Strategy B: Trained with all augmented data,

Strategy C: Trained with a data of 66% augmentation chance for each method.

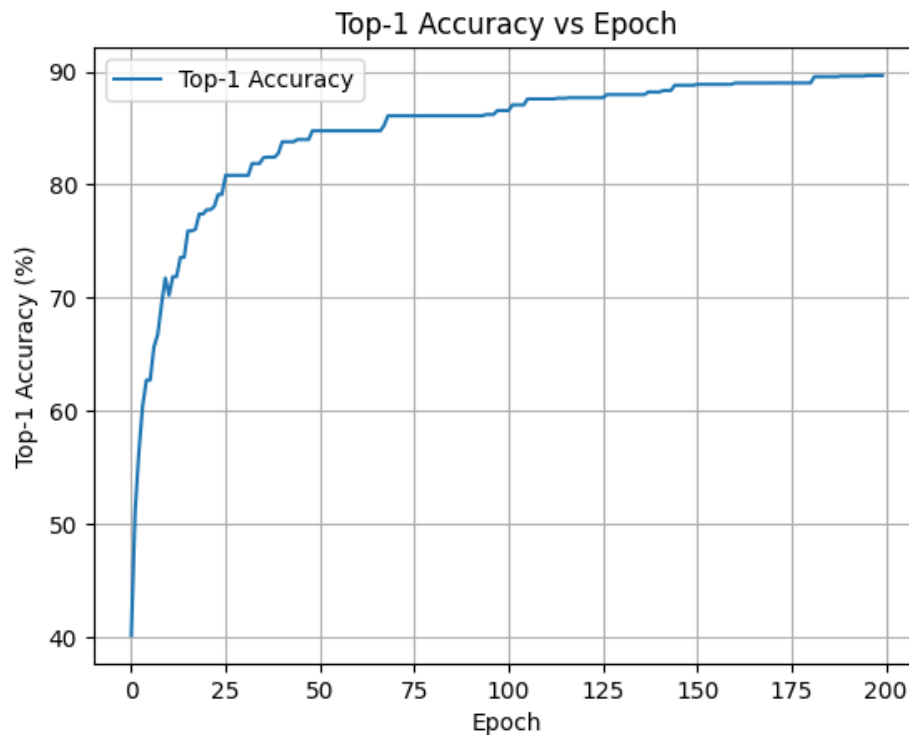
Former DataLoader: Augmentation applied to test dataset.

Updated DataLoader: No augmentation on test dataset.

	Former DataLoader	Updated DataLoader
--	-------------------	--------------------

Strategy A	84.751 (with 3x data)	-
Strategy B	90.820	90.763
Strategy C	91.117	92.810

These results shown in the table below. In addition, the Top-1 accuracy versus epoch graph of the Model C is plotted.



Energy Detector

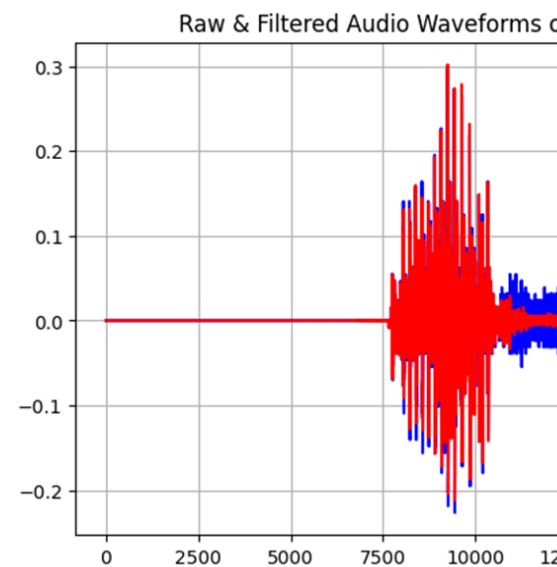
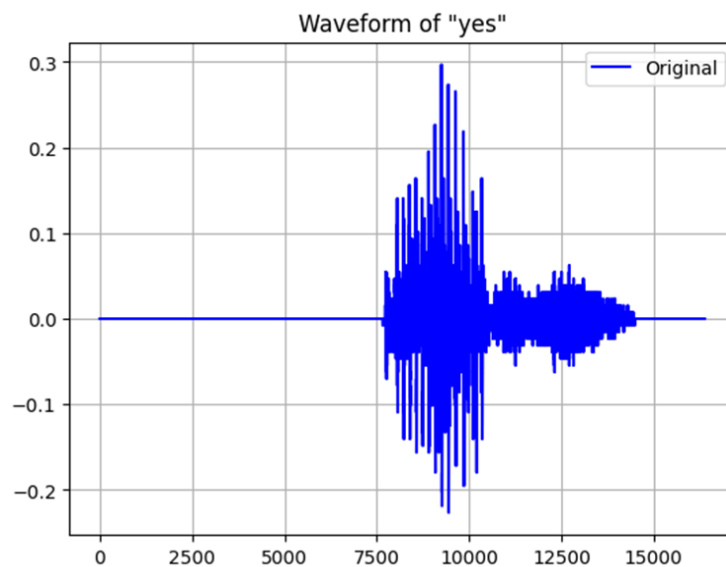
The position of the keyword within the audio must be taken into account in order to address the issue of time stretch and time shifts in our data augmentation method. The current method uses a cyclic time shift and ignores the position of the keyword, which can result in undesirable transformations. It could, for instance, alter the keyword "Backward" into "Ackward...B," which would be detrimental to both training and testing. This needs to be fixed by using the "energy/utterance detector" to identify keyword positions and implementing checks and limits during augmentation to stop problems like these.

Filtering Noise with Fast Fourier Transform (FFT)

The issues with shifting and stretching in the augmentation process are caused by the possibility of information loss, especially when these operations are not done carefully. Data loss and distortions can result from stretching operations that are applied carelessly or from excessively shifting audio signals.

A broad strategy is used to address these problems. The incoming signal is first given the Fast Fourier Transform (FFT), and a threshold is established based on the mean magnitude of the signal. Setting values below the threshold to zero helps to smooth the signal. The signal is then reconstructed in the time domain using the Inverse Fast Fourier Transform. The smoothed signal's mean magnitude is used to establish a new threshold. Based on this threshold value, the start point is then found. The signal is also flipped, and the same threshold value is used to determine the end point.

By carefully identifying the start point with $(\text{start_point} / \text{sampling_frequency}) * (-1)$ and end points with $(\text{end_point} / \text{sampling_frequency})$ of the audio signal, this method prevents information loss and guarantees accurate data augmentation.



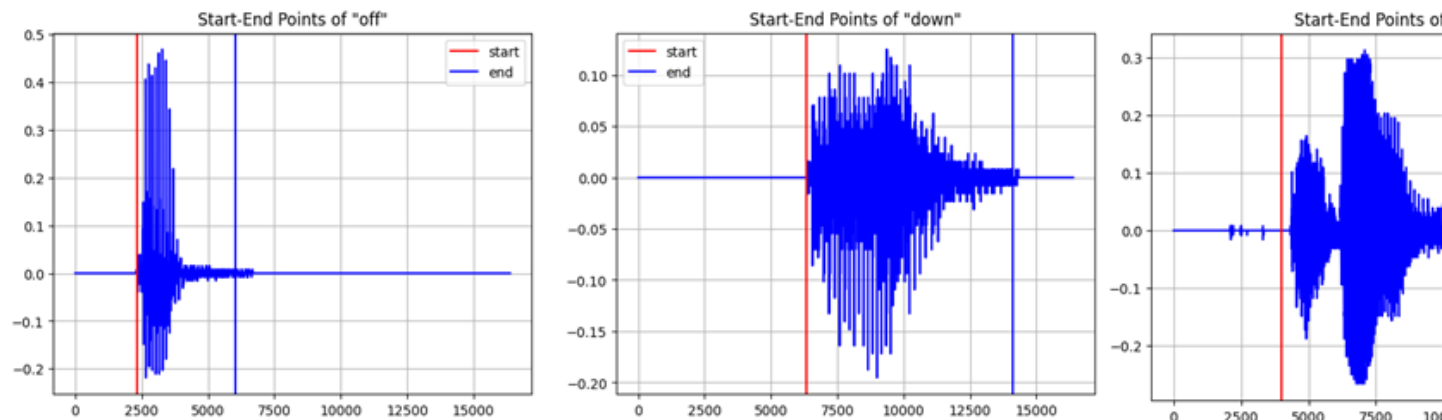
Improved Maximum Shift/Stretch Operations with Start/End point Determination

To effectively enhance the audio data, several crucial steps are involved in the augmentation process. The audio file first undergoes a stretch operation to lengthen its duration. An energy detector is then used to determine the characteristics of the resulting signal. The maximum permitted left and right shifts are established based on the start and end points of the audio, respectively, to introduce timing variations. The signal is subjected to the desired shifting using a

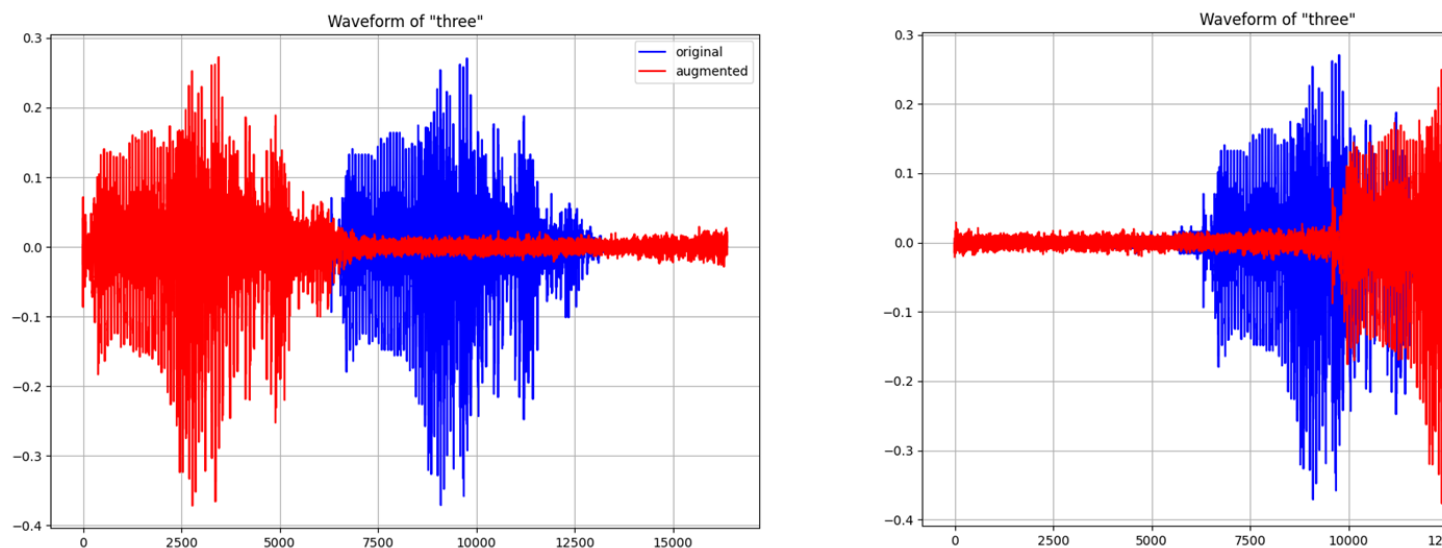
random coefficient generated within this range. In order to mimic real-world audio conditions, noise is also added.

The modified audio must then be transformed into a standard (128,128) format in order to meet the model's input specifications. The resulting (128,128) data, along with its associated label, is then ready for further processing or analysis.

There are some signals with the determined start end points below.



Min and max shifting is applied according to detected start-end points of the audio with previous noise and stretch coefficients.



Enhancing Data Splitting Strategy for Consistent Results

Splitting the data into training, validation, and test sets has been done in the current DataLoader by applying a completely random shuffling when the dataset is downloaded to a new folder. This random shuffling has significantly contributed to the variability in our results across various workstations, and possibly causing overfitting of the model. To address this issue, we have implemented a rewrite of this section to adhere to the training, validation, and test splits recommended by Google Speech Commands (GSC).

Noise Robustness Testing Results and Strategy Comparison

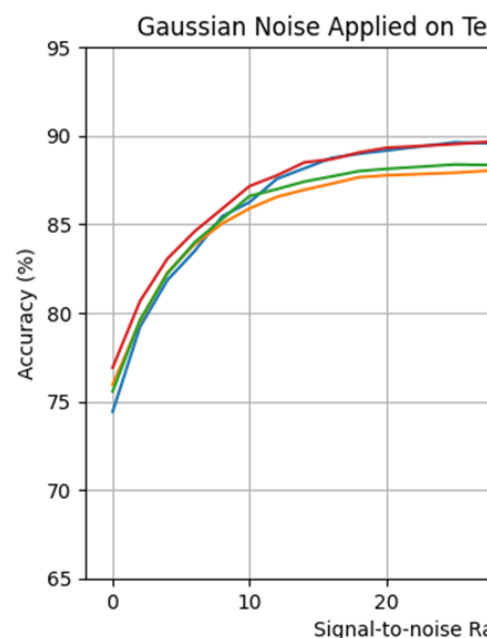
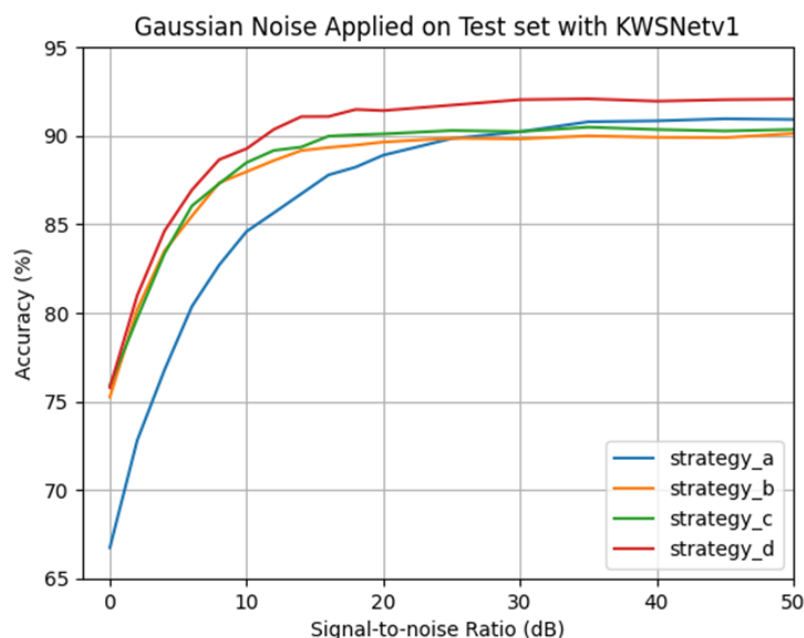
In our comprehensive noise robustness testing, we assess the performance of four different strategies tailored to various noise conditions and signal-to-noise ratios (SNR). The noise samples consist of Gaussian noise, and the Microsoft Scalable Noisy Speech Dataset (MS-SNSD) classes are evaluated for the KWS20Net_v1 and KWS20Net_v3 models with and without the GSC split. This comprehensive analysis provides insights into the overall effectiveness of these strategies across different noise types and model configurations.

Updated Strategies to cover:

- Strategy A: Existing Version
- Strategy B: Dynamic Augment
- Strategy C: Dynamic Augment + Energy Detector
- Strategy D: Dynamic Augment + Energy Detector + 66% Augmentation chance

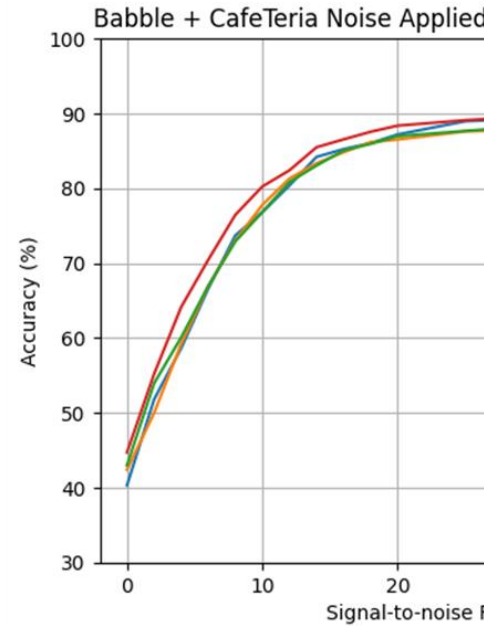
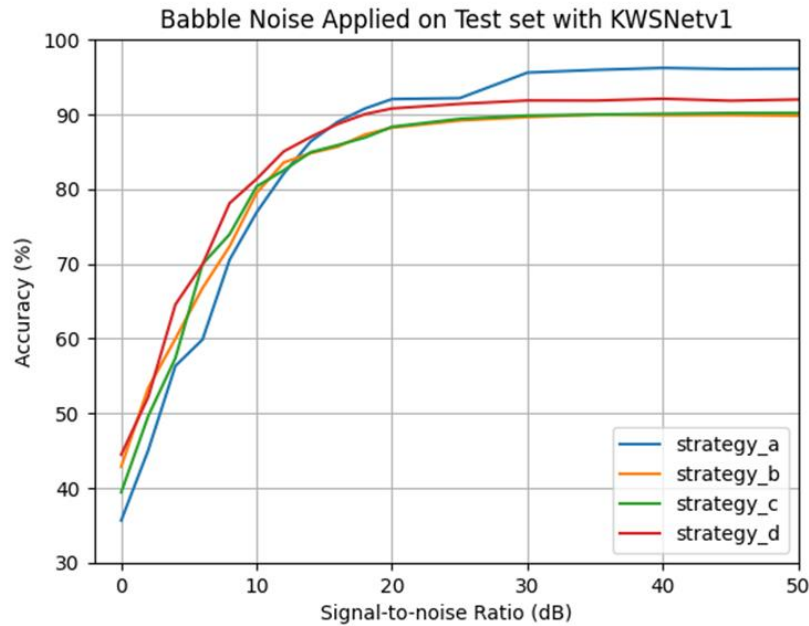
Comparing Gaussian Noise Results

We start by assessing these strategies in Gaussian noise conditions, evaluating accuracy scores for both the KWS20Net_v1, and KWS20Net_v3 model with GSC split. Across a range of SNR values, our results consistently demonstrate that Strategy D outperforms the others. Furthermore, Strategy D's accuracy increases steadily as SNR increases, demonstrating its resilience to Gaussian noise.



Comparing Different Noises of Microsoft Scalable Noisy Speech Dataset (MS-SNSD)

Shifting our focus to diverse noise types that belongs to MS-SNSD, we compare results under Babble noise for KWS20Net_v1 and Babble+CafeTeria mix noise for KWS20Net_v3 with the GSC split. Strategy D continues to excel in these challenging noise scenarios, consistently achieving the highest accuracy scores. This underscores Strategy D's adaptability to both Gaussian and complex, real-world noise conditions, emphasizing its robustness and versatility across various noise types and model configurations.



Conclusion

In conclusion, this project aimed to enhance the accuracy and robustness of the Keyword Spotting (KWS) demo application running on MAX78000/2 microprocessors. Several key improvements were made to various aspects of the application, including the data loader, data splitting strategy, data augmentation, and noise robustness testing. The transition to a dynamic data loader allowed for real-time augmentation and randomization of the dataset, resulting in improved model training and performance. The incorporation of an energy/utterance detector addressed issues related to time stretch and time shifts during data augmentation, ensuring more precise keyword positioning.

In addition, the noise robustness testing showed that Strategy D consistently outperformed other strategies in a range of noise conditions, from low to high signal-to-noise ratios. In particular, Strategy D clearly outperformed the current strategy, Strategy A, achieving roughly 3% to 4% higher accuracy in low SNR values.

These improvements have significantly improved the KWS demo application's accuracy and dependability and increased its ability to adapt to various noise types and real-world scenarios.